Time & Space Complexity and Asymptotic Notations

DSA lecture notes (from the YouTube lecture, course “Complete DSA from A to Z”). The lecture is in Hinglish; these notes are in simple English with the key terms kept as in the lecture.

In one minute

- An **algorithm** is a set of steps to solve a problem. DSA is about solving problems **effectively** (correctly) and **efficiently** (using little time and memory).
- **Time complexity** = how the number of instructions executed *grows as the input size n grows*. **Space complexity** = how much memory the code needs.
- We compare algorithms with **asymptotic notations**: **Big O** (upper limit / worst case), **Omega** (lower limit / best case), **Theta** (in between / average case).
- In practice you will almost always see and use **Big O**.

1. Why complexity? The cooking analogy

A recipe is an algorithm: it has a number of steps (instructions). If you follow them, the food is cooked. To make the recipe **efficient** we look at two things:

Factor	Question it answers	Efficient means
Time complexity	How much time does the algorithm take?	As little as possible
Space complexity	How much memory does the code use while running?	As little as possible

2. Time complexity vs execution time

Time complexity is **not** the time in seconds. Seconds depend on your machine. Time complexity counts **how many times the instructions execute** and how that count changes when the input changes.

	Execution time (seconds)	Time complexity
Depends on	CPU speed (old dual-core vs new processor), RAM / virtual memory, other programs running	Only the algorithm and the input size n
Measures	How long it took on this computer	How many times statements run, as n grows
Useful for	Benchmarking one machine	Comparing algorithms in general

Definition to remember (also for interviews): Time complexity tells us how much time our code takes to run **with respect to the input size**. If you leave out “input size”, the definition is incomplete.

Space complexity: how much memory (space) the code uses to do its task.

3. Why asymptotic notations?

- **To compare efficiency.** Suppose two algorithms do the same job, for example linear search (S1) and binary search (S2). Which is better? We check how their time grows as the input grows.
- **They are mathematical tools.** They let us **estimate the time from the input size without running the code**.

- **They describe behaviour for large inputs.** For 3 or 4 elements any algorithm is fine. With 10,000 or 10 lakh elements, the choice matters a lot.
- We care about the **rate of growth**: how the time grows as input grows, not "10 seconds vs 15 seconds".

Best, worst and average case: for the same algorithm the work done can differ depending on the data. Best case = least work, worst case = most work, average case = somewhere in the middle.

4. The three notations

Notation	Meaning	Case	Bound	Math definition
Big O O(g(n))	The algorithm will not take **more** than this	Worst case	Upper limit	$f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
Omega Ω(g(n))	The algorithm will not take **less** than this	Best case	Lower limit	$f(n) \geq c \cdot g(n)$ for all $n \geq n_0$
Theta Θ(g(n))	Stays **between** the lower and upper limit	Average case (as in the lecture)	Both limits	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$

Here **f(n)** is the actual time taken for input size n, **g(n)** is the simple comparison function (like n or n^2), and **c, c_1, c_2, n_0** are positive constants ($c > 0$, $n_0 > 0$).

Reading the graph (x-axis = input size n, y-axis = time):

- **Big O:** draw the curve $c \cdot g(n)$. After some point n_0, the real curve f(n) always stays **below** it. It may wiggle, but it never goes above. So $c \cdot g(n)$ is the upper limit.
- **Omega:** draw $c \cdot g(n)$ lower down. After n_0, f(n) always stays **above** it. It never goes below, so this is the lower limit (best case).
- **Theta:** draw both. f(n) is squeezed between $c_1 \cdot g(n)$ (below) and $c_2 \cdot g(n)$ (above) once $n \geq n_0$. This needs three constants: c_1, c_2 and n_0.

> **Most important point:** we focus on the worst case, so you will almost always see **Big O**. Omega and Theta appear rarely, but exams and interviews may ask you what best, worst and average case mean and which notation shows each.

A note on precision

> The lecture simplifies things, which is fine for learning. Strictly speaking: **Big O** is an upper bound, **Ω** is a lower bound, and **Θ** is a *tight* bound (both at once). Each can describe best, worst or average case behaviour. "Big O = worst case, Ω = best case, Θ = average case" is the usual way to remember them and is what most courses and interviews expect, but you can mention this detail if asked in more depth.

5. Example: linear search

Array: `[3, 5, 9, 2, 1]`. Linear search checks the elements one by one from the left.

Case	Situation	Comparisons	Notation
Best	Search for **3**: found at the first position	1 (constant)	Ω(1)
Worst	Search for **1**: found at the last position (or not present), so every element is checked	n	O(n)
Average	Search for **9**: found somewhere in the middle	about n/2, which still grows linearly	Θ(n)

Analogy: you gave a pen to one student in a class and have to find it. **Best:** the first student you ask has it. **Worst:** the last student has it, so you ask everyone. **Average:** someone in the middle has it.

6. Common growth rates (from best to worst)

Complexity	Name	How it grows	Verdict
O(1)	Constant	Same time however big the input is (one statement runs)	Best
O(log n)	Logarithmic	Grows very slowly; much better than linear	Excellent
O(n)	Linear	Input 10 gives about 10 steps, input 100 gives about 100	Good
O(n log n)	Log-linear	Between linear and quadratic	Acceptable
$O(n^2)$	Quadratic	Input 2 gives 4 steps, 4 gives 16, 100 gives 10,000	Poor for large n

The lecture says the curve for quadratic time shoots up very quickly. With 10,000 inputs, a quadratic algorithm needs about 100 million steps. The next lecture covers these common complexities in detail.

7. Key points to remember

- Complexity is about the **rate of growth** of time (or space) as the **input size** grows, not about seconds.
- Time complexity looks at **how many times statements execute**; execution time also depends on the CPU and RAM.
- Efficient algorithm = low time complexity **and** low space complexity.
- Big O = upper bound (worst case); Ω = lower bound (best case); Θ = between both (average case).
- You can compare algorithms mathematically **without running the code**.
- Big O is the one you will use most.

8. Likely interview questions

Q. What is time complexity?

How the time taken by the code grows with respect to the input size. It counts how many times instructions execute, not seconds.

Q. What is space complexity?

How much memory the code uses while it runs.

Q. Why is execution time not the same as time complexity?

Execution time depends on the machine (CPU, RAM). Time complexity depends only on the algorithm and input size.

Q. Name the three asymptotic notations.

Big O (upper bound / worst case), Omega (lower bound / best case), Theta (between both / average case).

Q. Which notation is used most, and why?

Big O, because we usually care about the worst case an algorithm can take.

Q. Give the best, worst and average case of linear search.

Best: element at the first position, Ω(1). Worst: last position or absent, O(n). Average: about the middle, Θ(n).

Next lecture: how to calculate time complexity and how to represent common complexities.